

Directional Optimal Execution Algorithm: LightGBM-Based Within-Minute Optimal Timing

Kai Liu, Mason Wong, Leif Fray Rogers, Patric Huang, Yung Hsueh Lee

Cornell University

May 12–13, 2026

- 1 Problem Setup
- 2 Feature Analysis
- 3 Algorithm Design
- 4 Training Label & Signal Design
- 5 Pipeline & Execution
- 6 Test Results (In-sample)
- 7 Test Results (Out-of-sample)
- 8 Conclusion

The Execution Problem

Goal: time one execution per minute per stock/side to beat the TWAP benchmark

- **Data:** 5-level LOB at 100 ms, AMZN, GOOG, INTC, MSFT — 270 minutes each
- **TWAP benchmark:** execute immediately at the first available ask (buy) or bid (sell)
- **Improvement metric:**

$$\Delta = \begin{cases} p_{\text{bench}} - p_{\text{exec}} & \text{buy (lower price = better)} \\ p_{\text{exec}} - p_{\text{bench}} & \text{sell (higher price = better)} \end{cases}$$

- **Round-trip cost reduction:** our primary aggregate metric; measures how much the total net (buy–sell) execution cost drops relative to TWAP

Theoretical Best (in hindsight)

Stock	Buy	Sell
AMZN	\$0.116/min	\$0.106/min
GOOG	\$0.219/min	\$0.171/min
INTC	\$0.010/min	\$0.007/min
MSFT	\$0.012/min	\$0.010/min

Best possible execution in hindsight (never achievable in practice)

Design Choice: 8 Independent Models

One LightGBM model per stock $\times$ side. Each model has its own hyperparameters, feature set, rules, and threshold — tailored to that stock's microstructure regime.

From Milestone 1 to Final Strategy

M1: ideas explored

Approach	Buy / Sell avg \$ (refined)
Spread threshold	0.0074 / 0.0125
Score-based rules	0.0004 / 0.0025
Cross-stock OFI	0.0060 / -0.0008
GRU prediction	0.0114 / -0.0063
PPO RL	0.0152 / -0.0024

Refined results: consistent 80/20 chronological split, no leakage, execute at ask/bid.

Common weakness across M1 approaches:

- One-size-fits-all rules failed INTC/MSFT (penny-spread stocks AMZN/GOOG)
- Sell side consistently weaker — signals were not symmetric
- Complex models (GRU, RL) had inconsistent convergence and poor out-of-sample sell performance

Three lessons that shaped the final design:

1 — Spread is powerful; go deeper, not broader

Spread is the strongest standalone signal. **Final:** 100+ LOB features with residualised depth/microprice; per-model importance selection keeps 8–9 that matter per stock/side.

2 — Abandon GRU & RL: complexity $\neq$ robustness

GRU/RL went **negative on sell** in every refined run; fragile convergence, high cost, minute-level overfit. **Final:** LightGBM on an intra-minute regret label — interpretable, stable, positive on both sides.

3 — Few strong signals beat many weak ones

Multi-signal combos noisier than spread alone. **Final:** stock/side-specific models; score-logic rules embedded *inside* LightGBM so the model learns when to trust them.

Data & Feature Engineering (100+ features)

Processing: 100 ms LOB $\xrightarrow{\text{resample}}$ 1-second bars
(10 ticks aggregated: last price, sum events)

Feature groups (100+ total):

- 1 Spread & liquidity (4)
- 2 Book pressure / imbalance (6)
- 3 Depth structure (21)
- 4 Price momentum & z-score (21)
- 5 Microprice & position-in-range (6)
- 6 Trade & cancel pressure (12)
- 7 Quote event counts (24)
- 8 Elapsed time since events (5)
- 9 Cross-stock signals (5)

Augmented features (engineered):

- 1 depth_slope_bid_resid (residualised)
 $\text{depth_slope_bid} - \text{OLS}(\cdot \text{depth_sum_bid})$
Isolates book-*shape* alpha from size level
- 2 microprice_edge_resid (residualised)
 $\text{microprice_edge_bps} - \text{OLS}(\cdot \text{book_pressure_1})$
Isolates tick-*position* alpha from imbalance level
- 3 pressure_term_structure (derived)
 $\text{book_pressure_1} - \text{book_pressure_3}$
Near-vs-far imbalance gradient; positive = support at L1
- 4 time_book_pressure3_interact (interaction)
 $\text{time_in_minute} \times \text{book_pressure_3}$ (MSFT)
- 5 time_ask_top_share1_interact (interaction)
 $\text{time_in_minute} \times \text{ask_top_share_1_of_5}$ (MSFT)

Feature Analysis: LightGBM Importance by Model

AMZN & GOOG — Spread-Dominated

Model	Feature	Gain (K)
AMZN buy	time_in_minute	227
	spread_bps	121
	spread_mean_roll_1s	118
	depth_ratio_5	40
AMZN sell	spread_bps	175
	spread_mean_roll_1s	101
	score_logic_signal	86
	momentum_10s_bps	72
GOOG buy	spread_bps	204
	depth_slope_ask	60
	ask_top_share_3of5	52
	momentum_10s_bps	51
GOOG sell	spread_bps	66
	time_in_minute	55
	momentum_10s_bps	43
	spread_mean_roll_1s	41

INTC & MSFT — Depth-Dominated

Model	Feature	Gain (K)
INTC buy	depth_slope_bid_resid	131
	BidSizeLog_1	126
	depth_ratio_5	111
	score_logic_signal	91
INTC sell	depth_sum_ask	207
	AskSizeLog_3	171
	depth_conc_diff_1_5	65
	momentum_10s_bps	40
MSFT buy	BidSizeLog_5	159
	time_in_minute	113
	ask_top_share_1of5	88
	book_pressure_3	69
MSFT sell	BidSizeLog_3	246
	book_pressure_3	88
	slope_diff_5	79
	score_logic_signal	66

Feature Analysis: Example of AMZN Buy — ICIR & Correlation Table

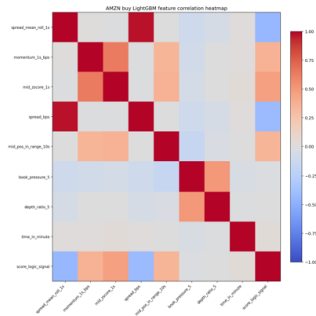
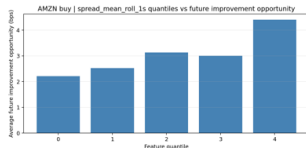
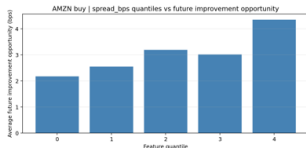
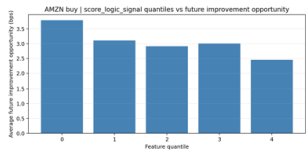
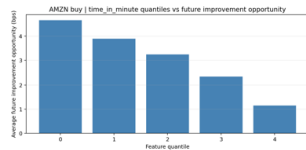
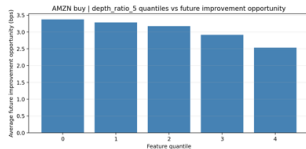
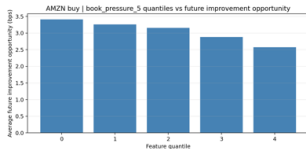
AMZN Buy: feature metrics from 216 valid training minutes

Feature	Spearman ρ	ICIR	Gain (K)	Splits	Interpretation
time_in_minute	+0.340	+0.755	227	246	Later → execute now (less time left)
spread_bps	−0.185	−0.447	121	170	Tight spread → execute
spread_mean_roll_1s	−0.182	−0.474	118	124	Sustained tight spread
depth_ratio_5	+0.072	+0.264	21	102	Bid-heavy book
book_pressure_5	+0.070	+0.256	16	73	Net buy pressure
score_logic_signal	+0.117	−0.077	20	71	Rule-based composite
mid_zscore_1s	+0.067	−0.115	8	46	Price above rolling mean
momentum_1s_bps	+0.067	−0.212	10	50	Short-term price uptick
mid_pos_in_range	−0.013	−0.587	17	92	Position in rolling range

Key insight: `time_in_minute` dominates (ICIR = 0.755).
Later in the minute, the current ask is likely the best remaining
— so the model learns to be more aggressive near second 59.

Negative ICIR for `momentum` and `mid_zscore`: these features may be useful locally (captured by LightGBM splits) but have noisy minute-level IC, showing LightGBM's advantage over linear models.

AMZN buy feature quantile plots / correlation heatmap



Why 8 Independent Models?

Regime 1: AMZN / GOOG — Spread-Driven

- High price; spread wide & volatile
- Execute when spread *compresses*; gate blocks wide-spread seconds
- Key signals: `spread_bps`, `time_in_minute`

Regime 2: INTC / MSFT — Depth-Driven

- Low price; penny tick, spread rarely changes
- Execute when *own-side depth* is large (queue support)
- Key signals: `BidSizeLog`, `depth_sum_ask`
- No spread gate; score logic off for INTC sell & MSFT buy

Execution timing (in-sample):

Model	Exec sec	Win rate	% of Oracle
AMZN buy	39.5	60.4%	33%
AMZN sell	26.9	50.0%	30%
GOOG buy	31.4	55.6%	39%
GOOG sell	31.9	54.4%	20%
INTC buy	20.5	34.4%	48%
INTC sell	17.7	34.4%	29%
MSFT buy	34.3	51.1%	53%
MSFT sell	15.6	38.5%	29%

Model hyperparameters — all 8 selected `depth4.balanced`:

Parameter	Value / Intuition
<code>max_depth=4</code> , <code>num_leaves=15</code>	Deep enough to capture $\text{spread} \times \text{time}$ and $\text{depth} \times \text{pressure}$ interactions
<code>lr=0.035</code> , <code>n_est=450</code>	Slow learning + many trees = stable fit
<code>reg_lambda=5</code> , <code>reg_alpha=0.75</code>	Strong regularisation on 270-min history
Threshold q^* (0.12–0.30)	Per-model; tuned on validation set

Training Label: Intra-Minute Regret Signal

Key question: at each second t within a minute, *how much better is executing now vs. waiting?*

Buy Target

$$y_t^{\text{buy}} = 10^4 \cdot \frac{\min_{s>t} p_{\text{ask},s} - p_{\text{ask},t}}{p_{\text{ask},t}}$$

- $y_t > 0$: current ask is cheapest remaining $\Rightarrow$ **execute now**
- $y_t < 0$: cheaper ask coming $\Rightarrow$ **wait**

Sell Target

$$y_t^{\text{sell}} = 10^4 \cdot \frac{p_{\text{bid},t} - \max_{s>t} p_{\text{bid},s}}{p_{\text{bid},t}}$$

- $y_t > 0$: current bid is highest remaining $\Rightarrow$ **execute now**
- $y_t < 0$: higher bid coming $\Rightarrow$ **wait**

Why this label?

- Directly measures intra-minute opportunity cost — the signal we want to maximise
- Constructed entirely from *point-in-time* data; future refers only to the same minute
- LightGBM learns to rank seconds: high-score second $\Rightarrow$ execute

Implementation (panel.py):

- Group by (stock, minute); work within each group independently
- Reverse-cummin (buy) or reverse-cummax (sell) to get future extremum
- Shift by -1 to get the best price *strictly after* the current second
- Fill last second with current price (target = 0

Score Logic Signal: Rule-Based Baseline as LightGBM Input Feature

Architecture: hand-crafted rule score *fed as a feature* into LightGBM

- 1 Calibrate quantile thresholds on training minutes for each rule
- 2 Score each second: $s_t = \sum_j w_j \mathbf{1}[\text{rule}_j(t)]$
- 3 Apply exponential-weighted smoothing (span = 3) within each minute
- 4 Pass result as `score_logic_signal` to LightGBM

Example — AMZN buy rules:

Feature	Condition	w
<code>spread_mean_roll_1s</code>	$\leq$ 20th pct	2.0
<code>momentum_1s_bps</code>	$\geq$ 80th pct	1.0
<code>mid_zscore_1s</code>	$\geq$ 65th pct	2.0

Interpretation: execute when spread compresses, short-term momentum is positive, and price sits above its recent mean.

Gate rules (hard execution block):

Model	Block condition
AMZN buy	<code>spread_bps</code> > 30th pct
AMZN sell	<code>spread_bps</code> > 35th pct
GOOG buy	<code>spread_bps</code> > 25th pct
GOOG sell	<code>spread_bps</code> > 25th pct
INTC/MSFT	<i>no gate rule</i>

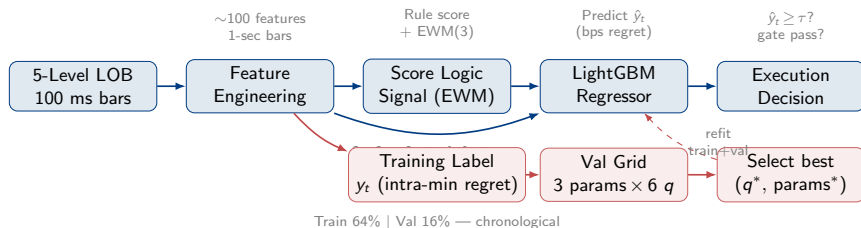
Disabled models:

`score_logic_signal` is *not* used for INTC sell and MSFT buy: depth features dominate those models and adding the spread-based rules adds noise.

Hybrid Insight

Rule-based signal provides a structured *prior*; LightGBM learns *when to trust it* and *when to override it* using depth/momentum context.

Full Pipeline Architecture



- **8 independent models** (4 stocks $\times$ 2 sides): own feature set, score rules, gate rules, threshold
- **Score logic signal** computed first, then passed as a feature into LightGBM
- **Final refit** on train + val minutes at the iteration count selected by the validation grid

Decision rule at each second t within a minute:

- 1 **Gate check:** if gate rule fails $\Rightarrow$ skip this second (no execution)
- 2 **Score:** compute $\hat{y}_t = \text{LightGBM}(\mathbf{x}_t)$
- 3 **Execute at first t with $\hat{y}_t \geq \text{quantile}(q)$ and gate passed**
- 4 **Fallback:** if no threshold crossed, execute at second 59 (*forced-last*)

Threshold calibration:

$$q \in \{0.08, 0.12, 0.16, 0.20, 0.24, 0.30\}$$

Selected on validation set. Higher q = more aggressive (executes earlier / more often).

Validation objective:

$$\mathcal{L} = \bar{\Delta} - 0.01 r_{\text{last}} - 0.05 \sigma_{\Delta}$$

$\bar{\Delta}$ = mean improvement, r_{last} = forced-last rate, σ_{Δ} = improvement std

Model	Params	q^*	r_{last}
AMZN buy	depth4_balanced	0.30	24%
AMZN sell	depth4_balanced	0.24	21%
GOOG buy	depth4_balanced	0.12	31%
GOOG sell	depth4_balanced	0.30	26%
INTC buy	depth4_balanced	0.30	11%
INTC sell	depth4_balanced	0.30	10%
MSFT buy	depth4_balanced	0.24	1%
MSFT sell	depth4_balanced	0.30	6%

In-Sample Performance (Full Training Set, 270 min/model)

Per-model improvement vs. TWAP

Stock	Side	Avg \$	Win%	Sec	Best%
AMZN	buy	0.038	60.4	39.5	33
AMZN	sell	0.031	50.0	26.9	30
GOOG	buy	0.085	55.6	31.4	39
GOOG	sell	0.034	54.4	31.9	20
INTC	buy	0.005	34.4	20.5	48
INTC	sell	0.002	34.4	17.7	29
MSFT	buy	0.006	51.1	34.3	53
MSFT	sell	0.003	38.5	15.6	29
All	buy	0.034	50.4	31.4	38
All	sell	0.018	44.4	23.0	24
ALL	ALL	0.026	47.4	27.2	32

Avg \$: mean per-trade improvement; Sec: avg execution second; Best%: % of theoretical best captured

Round-trip cost reduction vs. TWAP

Stock	Algo (\$)	TWAP (\$)	Saving
AMZN	19.08	37.86	49.6%
GOOG	46.67	78.93	40.9%
INTC	0.85	2.75	69.0%
MSFT	0.24	2.75	91.4%
ALL	66.84	122.29	45.3%

Net cost = total buy spend – total sell proceeds.
Saving = (TWAP – Algo) / TWAP.

Key takeaway

Algo saves **\$55.45** of the \$122.29 round-trip cost that TWAP incurs — a **45%** reduction.
INTC/MSFT savings highest (%) because first-tick TWAP is particularly wasteful on penny-spread stocks.

Out-of-Sample Test Results (4 Stocks, 120 Test Minutes)

Per-model test performance

Stock	Side	Avg \$	Win%	% vs TWAP
AMZN	buy	0.018	54.2%	31.5%
AMZN	sell	0.018	48.3%	
GOOG	buy	0.037	45.8%	30.5%
GOOG	sell	0.030	44.2%	
INTC	buy	0.001	19.2%	11.0%
INTC	sell	0.000	25.8%	
MSFT	buy	0.003	51.7%	50.4%
MSFT	sell	0.002	30.0%	
ALL	buy	0.015	42.7%	30.9%
ALL	sell	0.012	37.1%	

Train vs. Test: % improvement vs TWAP

Stock	Train	Test	Δ
AMZN	49.6%	31.5%	−18pp
GOOG	40.9%	30.5%	−10pp
INTC	69.0%	11.0%	−58pp
MSFT	91.4%	50.4%	−41pp
ALL	45.3%	30.9%	−14pp

Key observations

- AMZN/GOOG: stable OOS (~30%); spread regime persists
- MSFT: strongest OOS (50%); depth + time signals generalise well
- INTC: largest drop (−58pp); depth regime may have shifted; model needs more calibration history
- No re-tuning after data release; all params frozen

AAPL: Autonomous Template Selection & Results

Autonomous selection protocol:

- 1 For each of 4 source templates (AMZN/GOOG/INTC/MSFT), train on AAPL training data using that template's feature set and score-logic rules
- 2 Evaluate on a held-out validation fold of AAPL training data
- 3 Select the template with the *highest validation objective*
- 4 Refit on full AAPL training data; apply to AAPL test set

Template selection result:

Side	Selected	Val obj	Runner-up
Buy	INTC	0.068	GOOG (0.041)
Sell	MSFT	0.055	INTC (0.055)

AAPL performance:

		Train	Test
Buy	Avg improv	\$0.079	\$0.019
	Win rate	61.5%	50.0%
Sell	Avg improv	\$0.067	\$0.011
	Win rate	55.9%	47.5%
% vs TWAP		87.1%	24.7%

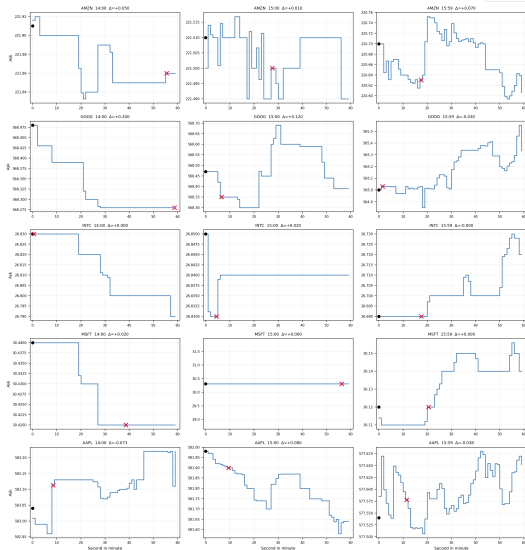
Discussion

Train numbers are now in-sample (threshold tuned on training data) and are inflated; the 24.7% OOS gain is the meaningful figure. Both buy and sell are positive OOS; the depth-driven INTC/MSFT templates outperform the spread-based GOOG template when the full calibration data is used.

Test-Set: Sampled In-Minute Execution (all 5 stocks)

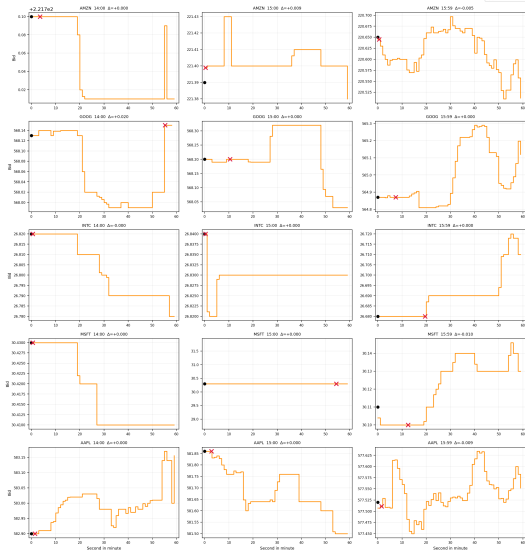
Test-Set Sampled In-Minute Buy Execution (all 5 stocks)

● THMP (sec 8)
✗ Model exec



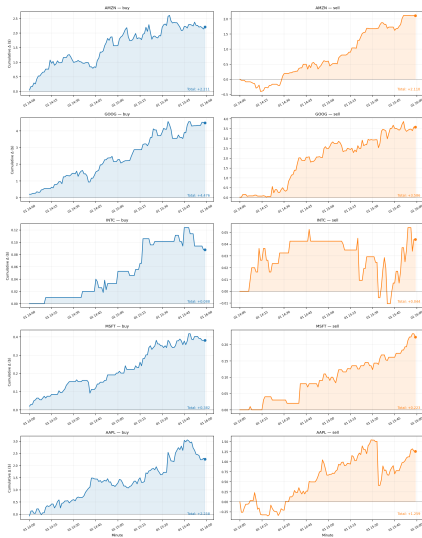
Test-Set Sampled In-Minute Sell Execution (all 5 stocks)

● THMP (sec 8)
✗ Model exec

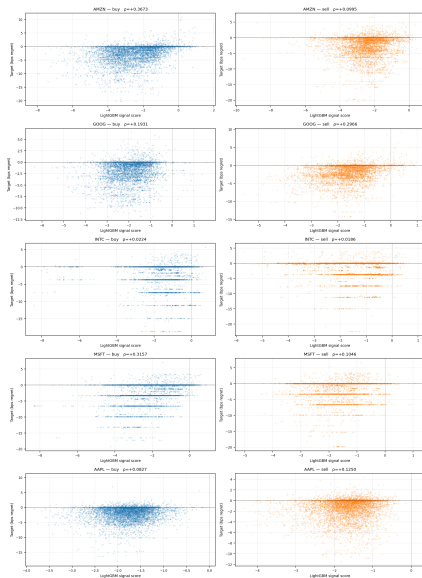


Test-Set: Cumulative Improvement & Signal-Target Correlation

Test-Set Cumulative Price Improvement vs TWAP (all 5 stocks)



Test-Set Signal-Target Correlation (LightGBM score vs intra-minute regret bps)



Conclusion & Key Takeaways

Key achievements:

- **29% test** round-trip cost reduction vs. TWAP across all **5 stocks** (AMZN / GOOG / INTC / MSFT / AAPL)
- **AAPL autonomous**: INTC/MSFT feature template selected without manual tuning; **+24.7% OOS**
- Captures **28%** of the theoretical oracle improvement
- Positive OOS performance for all 5 stocks

Bottom Line

LightGBM with stock/side-specific microstructure features and a hybrid rule + ML signal **beats TWAP by 29% out-of-sample** across all 5 equities, including an autonomous extension to AAPL with no manual parameter changes.

Reflections:

- **In-sample overfitting**: all models show a train → test drop (45% → 29% overall); threshold calibrated on training data inflates in-sample metrics — OOS is the meaningful benchmark
- **INTC covariate shift**: BidSizeLog₋₁ was $\sim 1.8\times$ larger in test, inflating scores and causing early misfires; depth models are sensitive to absolute LOB scale
- **Regime generalisation**: spread-driven models (AMZN/GOOG) more robust OOS; depth-driven (INTC/MSFT) more sensitive to distributional stability
- **Score logic as feature**: LightGBM learns *when* to trust the rule signal — outperforms hard gating alone

Future directions

Rolling threshold recalibration; feature distribution monitoring (PSI) for covariate-shift detection; larger training window for new-stock bootstrapping